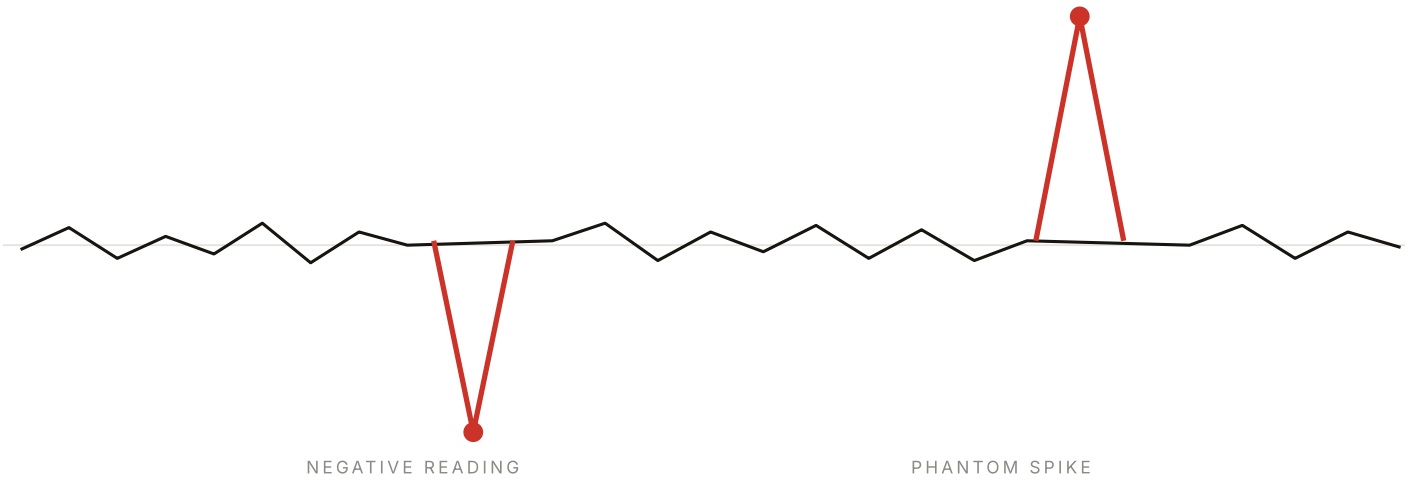
GreenGrid

Nº01

TELEMETRY · DECEMBER 2025



A Databricks Lakehouse medallion pipeline. Bronze ingestion, a Silver quality firewall with SHA-256 anonymisation, and Gold aggregation, for a smart energy pilot at one hundred residential nodes.

The Smart Pricing *Pilot*

CONTENTS

Six sections.

One medallion pipeline that makes the pilot bill. A hundred nodes, a month of telemetry, and three layers from raw to gold.

01 Executive Brief & Client Overview — P.03–04

Situation, complication, resolution. Why one hundred homes gate the Series B and the fifty-thousand rollout.

02 Problem Diagnosis — P.05

Noisy sensors, manual Excel logic, raw PII on a shared drive. Three failures that compound into a liability.

03 The Medallion Architecture — P.06

Bronze, Silver, and Gold on Databricks. Validation upstream, privacy into the schema, aggregation for reporting.

04 Engineering Mandate & Schema — P.07–08

The four notebooks, the source exhibit, and the tables each layer writes.

05 Acceptance Criteria — P.09

Completeness, integrity, governance, peak logic. The gates the build must pass.

06 Discussion, Lab & Orchestration — P.10–12

Five questions, the four-notebook build, and the scheduled job that runs it.

EXECUTIVE BRIEF

Stop billing from spreadsheets.

GreenGrid must move from manual, spreadsheet billing to an automated, auditable Databricks pipeline. The pilot of one hundred nodes gates the Series B and the Q3 rollout to fifty thousand homes.

One hundred suburban properties (meters M-101 to M-200) on a Time-of-Use model. Peak runs six to nine in the evening at a doubled rate, proving the dynamic model before the Q3 expansion.

Noisy telemetry: negative readings and phantom nine-thousand-kilowatt spikes. Excel cannot filter them, so wrong bills reach customers. Emails sit in plain text, joined to usage, flagged non-compliant.

A medallion pipeline on Databricks. Bronze ingests raw, Silver quarantines the anomalies and hashes the PII with SHA-256 before billing, Gold aggregates for reporting. No raw email crosses into analytics.

It is a question of order. Ingest, then quarantine, then transform and hash, then aggregate. Reverse the sequence and the pilot ships the very mistake it was built to prevent.

If we send one wrong bill during this pilot, customer trust is gone forever. I cannot bill someone five thousand dollars for a toaster glitch.

HEAD OF PRODUCT · GREENGRID

CLIENT OVERVIEW

A hardware startup, becoming a data enterprise.

Its Smart Energy Gateway captures consumption through the day, charging premium at peak and rebating surplus renewable supply. The pilot is the first field test.

FROM HARDWARE TO DATA

The shift to a data enterprise has exposed fractures in operational maturity. The infrastructure was never designed for scale, and raw IoT logs reach invoices through manual exports, CSV, and Excel.

THE BOARD MANDATE

Investors flagged Billing Integrity and Data Privacy as high-risk. The Board directive: the platform must mature from experiment to enterprise utility immediately. The pilot cannot proceed as built.

15%

Failure rate in Time-of-Use calculations

$12K

Monthly revenue leakage per 100 homes

A

Class A liability from unencrypted PII, blocking Q3

KEY METRIC	CURRENT PILOT	TARGET PRODUCTION
Network Scale	*100 residential nodes*	50,000 nodes by Q3
Billing Latency	*72 hours of manual preparation*	Under 60 minutes, automated
Data Governance	*Ad-hoc, PII in plain text*	Zero Trust, PII hashed at ingestion

PROBLEM DIAGNOSIS

Three failures. One compliance gate.

A forensic review reveals a systemic breakdown in the last mile. The path from raw telemetry to financial record relies on human-in-the-loop processing, and it breaks at every layer that matters.

FAILURE 01

62

Unclean Telemetry

Negative readings from interference, and transient spikes registering as nine-thousand-kilowatt events in a single window. The month carries sixty-two such anomalies.

FAILURE 02

$12K

Manual Logic Gap

Excel fails to apply Peak Hour multipliers consistently. The result is silent leakage, undercharging during the most expensive congestion windows.

FAILURE 03

4%

PII Exposure

Plain-text emails on a shared drive, joined directly to usage. Shadow copies proliferate across analyst desktops.

THE MEDALLION ARCHITECTURE

Raw, refined, *reported.*

The strategy is a three-layer Lakehouse on Databricks. Each layer has one job, reads the layer before it, and writes its own Delta tables partitioned by processing date.

RAW CAPTURE → QUALITY, PRIVACY, BILLING → AGGREGATION



ARCHITECTURAL PRINCIPLE

Quality is enforced before billing logic sees a value. Hashing is applied before any join result is written. The order is non-negotiable: ingest, then quarantine, then transform and hash, then aggregate. Reversing it is the entire problem this architecture exists to solve.

ENGINEERING MANDATE

You are the Lead Engineer.

Build four notebooks, one per stage. Each reads the previous table and writes its own, driven by a processing_date widget so the job can run any day. The code you write determines the accuracy of every invoice.

EXHIBIT 1

Source: *meter_readings.json*, 2025-12-18

NON-NEGOTIABLE

Readings outside zero to one hundred kWh are quarantined before billing runs. Emails never persist in Silver. Unhashed PII in production halts the pilot.

METER	TIMESTAMP	KWH	STATUS
M-101	2025-12-18T14:00:00	2.5 · standard rate, off-peak	OK
M-101	2025-12-18T19:30:00	3.0 · peak rate, inside 18:00 to 21:00	OK
M-102	2025-12-18T08:00:00	-10.0 · negative, quarantined as negative_reading	ERR
M-103	2025-12-18T20:00:00	9000.0 · over_ceiling, status flag unreliable	OK

TARGET SCHEMA

The Silver contract.

~~CustomerEmail~~

DROPPED BEFORE WRITE ↓

CustomerHash

Every column has a derivation rule. One is conspicuously absent: there is no place for a raw email address.

COLUMN	TYPE	DERIVATION LOGIC	CONSTRAINT
meter_id	String	Direct mapping from the bronze telemetry record.	PASS-THROUGH
kwh	Double	Source value after the quality filter. Above zero, below one hundred.	RANGE GATE
CustomerHash	SHA-256	Hash of the email. Raw column dropped before write.	NO RAW TEXT
FinalRate	Double	Base rate, doubled by peak_multiplier when the hour is in the 18 to 21 window.	CONDITIONAL
Bill_Amount	Double	kwh multiplied by FinalRate, after gate and rate resolve.	CALCULATED

EVERY TABLE ALSO CARRIES

processing_date (the partition key), *ingested_at*, and *run_id*, so every row traces back to the exact run that produced it.

QUARANTINE ADDS

a *rejection_reason* of *negative_reading* or *over_ceiling*, with the vendor *status* kept alongside for cross-check.

ACCEPTANCE CRITERIA

Four gates. Pass all, or it does not resume.

Evaluated against strict operational and compliance benchmarks. Failure of any criterion fails the deployment, and the gates are not graded on partial credit.

01 Completeness

Bronze ingests one hundred percent of files for the run date. No usage data orphaned or unbilled.

02 Integrity

Any reading of zero or below, or one hundred or above, is routed to *quarantine_readings* with a rejection_reason.

03 Governance

Silver_SmartBill stores no raw email. Every identifier is transformed through SHA-256.

04 Peak Logic

Readings at or after 18:00 and before 21:00 carry a doubled rate in FinalRate.

VALIDATION

Run for 2025-12-18. The quarantine holds exactly two rows, the -10.0 from M-102 (negative_reading) and the 9000.0 from M-103 (over_ceiling). CustomerHash shows no email pattern, and M-101 at 19:30 bills to 1.50.

DISCUSSION

Five questions. Reasoning, not recollection.

Q1 Complexity vs compliance

Why hash the email rather than delete it? Reference referential integrity, and explain how a hashed identifier preserves it while satisfying the Chief Legal Officer.

Q2 Business logic validation

Calculate the Bill_Amount for M-101 at 2025-12-18T19:30:00. Show each step: plan, base rate, multiplier, final value.

Q3 Layer sequencing

Silver quarantines before it bills. What happens to Bill_Amount if the order reverses and the -10.0 record reaches billing? Silent error or failure, and why is either unacceptable?

Q4 Scalability

At one hundred thousand homes, what changes in the PySpark logic, and why does distributed compute handle this differently from a row-by-row model?

Q5 Platform trade-offs

Why a PySpark notebook over a low-code, SQL-only approach? Cite a concrete transformation step from the Silver layer.

LAB INSTRUCTIONS

Four notebooks. One per layer.

Build in order. Each notebook declares a processing_date and run_id widget, reads the previous layer, and writes Delta partitioned by date with replaceWhere so a re-run is safe.

NOTEBOOK	OPERATION	OUTPUT TABLE
01_bronze_ingest	Read the three sources, filter readings to the date, write raw.	BRONZE_*
02_silver_quarantine	Split out-of-range rows, tag rejection_reason, write.	QUARANTINE
03_silver_smartbill	Re-filter clean, join, hash the email, apply the peak rate, bill.	SILVER
04_gold_daily_usage	Aggregate peak and off-peak totals per meter per day.	GOLD

REMINDER

Ingest first, quarantine second, hash third, aggregate last. The order is the entire engineering discipline this case exists to teach.

ORCHESTRATION

One job runs *all four.*

The four notebooks become four tasks in a Databricks Job. Job parameters set the date, multiplier, and run id once, and every task receives them through its widgets.

PARAMETERS, SET ONCE

processing_date, *peak_multiplier*, and *run_id* are defined at the job level and mapped into each task's widget, so the schedule feeds them with no manual input.

TASK CHAIN

Bronze runs first. Quarantine and SmartBill both depend on Bronze. Gold depends on SmartBill. A daily schedule fills the date; backfilling a past date re-runs safely because replaceWhere replaces only that day.

4

Tasks, one per medallion notebook

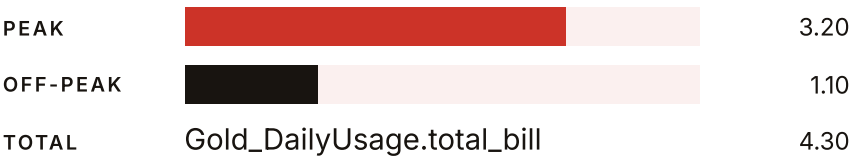
31

December dates a backfill can replay

0

Manual inputs once the schedule runs

SAMPLE GOLD OUTPUT · ONE METER, ONE DAY



IDEMPOTENCY

Every write is Delta, partitioned by processing_date, using replaceWhere on that date. Re-running any day replaces only its own rows and leaves every other day untouched, which is what makes the daily schedule and the backfill safe.

From noisy telemetry to audit-ready bills at scale.

A bronze layer that captures, a silver firewall that quarantines and anonymises, a gold layer that reports. One scheduled job, any date, one trusted output the pilot can resume against.

WHAT YOU DELIVER

Four Databricks notebooks (Bronze, Silver quarantine, Silver SmartBill, Gold), parameterised by widgets and run as a scheduled job. Five Delta tables across the three layers.

WHAT IT PROVES

You can layer a medallion pipeline, move quality enforcement upstream of billing, apply anonymity without breaking referential integrity, and make every run idempotent.

HOW YOU ARE EVALUATED

Run for 2025-12-18: the quarantine holds M-102's -10.0 and M-103's 9000.0, each with a rejection_reason. CustomerHash shows no email pattern. M-101 at 19:30 bills to 1.50.

WHAT COMES NEXT

Power BI on Gold_DailyUsage. Unity Catalog masking on CustomerHash. A salted hash, and an alert channel on the quarantine. The pilot scales to fifty thousand homes without re-architecture.

INSTRUCTOR · SURYA KUMAR